VERIQTA

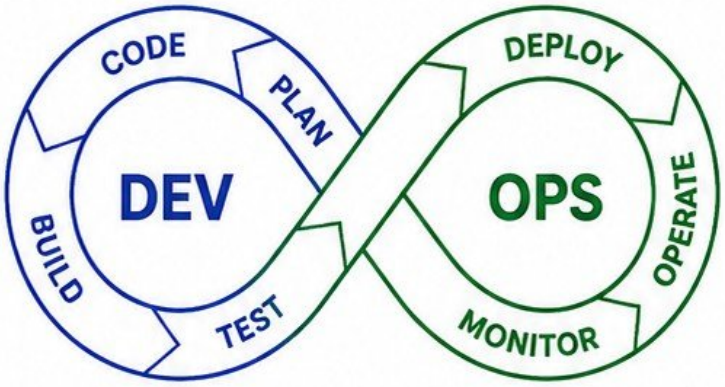
FIRST 90 DAYS AS A DEVOPS ENGINEER

YOUR 90-DAY ROADMAP TO LEARN, CONTRIBUTE, AND OWN PRODUCTION

AUTOMATE

CONTAINERIZE

ORCHESTRATE



CLOUD

MONITOR

SECURE

- Understand the Environment
- Learn the Tools and Processes
- Observe and Ask Questions
- Solve Real Problems
- Automate and Improve
- Own Production with Confidence

AWS | DOCKER | KUBERNETES | TERRAFORM | NGINX | PROMETHEUS | GRAFANA | GITHUB

VERIQTA

/VERIQTA /VERIQTA /VERIQTA /VERIQTA /VERIQTA /VERIQTA

VERIQTA

YOUR FIRST 90 DAYS

AS A DEVOPS ENGINEER

WHAT COMPANIES ACTUALLY EXPECT FROM A NEW DEVOPS ENGINEER

- Be reliable and easy to work with
- Learn the systems and processes
- Understand the business and customer impact
- Improve safety, stability, and developer productivity
- Take ownership and communicate clearly

LEARN BEFORE CHANGING

- Listen more than you speak
- Understand the history behind current decisions
- Read documentation and runbooks
- Ask questions and take notes
- Observe real-world operations

UNDERSTAND THE ENVIRONMENT BEFORE TOUCHING PRODUCTION

- Map the architecture and dependencies
- Learn the deployment process
- Understand monitoring and alerts
- Know the rollback and recovery plan
- Avoid making changes until you understand the impact

BUILD TRUST WITH DEVELOPERS AND OPERATIONS TEAMS

- Be supportive and approachable
- Respect everyone's expertise
- Communicate clearly and often
- Share knowledge and document your learning
- Deliver small wins and keep promises

30, 60, 90-DAY PROGRESSION

30 DAYS — UNDERSTAND

- Learn the team and culture
- Understand systems and tools
- Map architecture and flows
- Read docs and runbooks
- Shadow deployments and operations

→

60 DAYS — CONTRIBUTE

- Take on small changes
- Improve monitoring and alerts
- Automate manual tasks
- Troubleshoot with the team
- Contribute to documentation

→

90 DAYS — OWN

- Handle deployments confidently
- Investigate and resolve issues independently
- Drive improvements
- Strengthen reliability and security
- Be a trusted production engineer

WHAT SUCCESS SHOULD LOOK LIKE BY DAY 90

- ✓ You understand the production environment
- ✓ You can deploy safely and roll back
- ✓ You troubleshoot production issues independently
- ✓ You know where to find key information fast
- ✓ You have improved something that matters
- ✓ You communicate clearly and proactively
- ✓ You are trusted with production responsibilities
- ✓ You think like an owner, not just an executor

PAGE 2

WEEK 1

UNDERSTAND THE ENGINEERING ORGANIZATION

YOUR FIRST PRIORITY: PEOPLE, ROLES, AND RESPONSIBILITIES

1. ENGINEERING TEAMS & OWNERSHIP BOUNDARIES

PRODUCT TEAMS	PLATFORM TEAM	DEVOPS TEAM	SRE TEAM	SECURITY TEAM
Own features and applications	Builds internal platforms and tools	Enables delivery, automation, and reliability	Ensures reliability, performance, and incident response	Protects systems, data, and compliance

KNOW WHO OWNS WHAT.
CLEAR BOUNDARIES PREVENT CONFLICT AND SPEED UP DELIVERY.

2. ROLES & RESPONSIBILITIES OVERVIEW

DEVELOPMENT	• Write and test code • Own application functionality • Fix bugs and ship features
PLATFORM ENGINEERING	• Build and maintain internal platforms • Provide self-service tools • Improve developer experience
DEVOPS ENGINEERING	• CI/CD pipelines and automation • Infrastructure as Code • Deployment and release management
SRE ENGINEERING	• Reliability and availability • Monitoring and alerting • Incident response and postmortems
SECURITY ENGINEERING	• Security policies and standards • Identity and access management • Vulnerability management

3. WHO OWNS PRODUCTION?

PRODUCTION IS A SHARED RESPONSIBILITY

- ✔ Application code → Development Team
- ✔ Infrastructure → DevOps / Platform Team
- ✔ Reliability → SRE Team
- ✔ Security → Security Team
- ✔ Change approvals → Defined by Org
- ✔ Incident response → On-call Rotation

FIND OUT WHO MAKES FINAL DECISIONS WHEN ISSUES AFFECT PRODUCTION.

4. ESCALATION PATHS

ENGINEER
↓
TEAM LEAD
↓
ENGINEERING MANAGER
↓
DIRECTOR / HEAD OF ENGINEERING
↓
CTO / VP ENGINEERING

- ✔ Understand the official escalation path
- ✔ Know who to page and when
- ✔ Know communication channels for each level
- ✔ Escalate early, not late
- ✔ Keep escalations clear, concise, and factual

5. TECHNICAL DECISION MAKERS

TECH LEADS	PLATFORM OWNERS	PRODUCT OWNERS	SECURITY OWNERS	FINOPS / CLOUD OWNERS
Architectural and technical direction	Tooling and standards	Priorities and roadmap	Risk and compliance	Cloud costs and budgets

IDENTIFY THE PEOPLE WHO CAN APPROVE, INFLUENCE, OR BLOCK CHANGES.

6. FIND THE PEOPLE WHO KNOW HOW SYSTEMS REALLY WORK

LONG-TIME ENGINEERS	ON-CALL ENGINEERS	BUILD & RELEASE OWNERS	SREs	OPERATIONS ENGINEERS
Know the history and gotchas	Know real issues and workarounds	Know pipelines and deployments	Know reliability metrics and incidents	Know infrastructure and dependencies

7. QUESTIONS TO ASK DURING YOUR FIRST WEEK

1 Who owns this service in production?	7 What monitoring and alerting exist today?
2 How does code go from commit to production?	8 What tools and accounts will I need access to?
3 What are the critical dependencies?	9 What changes require approvals and from whom?
4 What are the biggest pain points or risks?	10 What is the release process and rollback plan?
5 How are incidents handled?	11 What good looks like in the next 30, 60, 90 days?
6 Where do I find runbooks and documentation?	12 Who should I build strong relationships with?

THE GOAL THIS WEEK

- ✔ Understand the org
- ✔ Map responsibilities
- ✔ Build relationships
- ✔ Learn the systems
- ✔ Ask smart questions
- ✔ Listen more, speak less

OBSERVE. LEARN. EARN TRUST.

PAGE 3

MAP THE PRODUCTION ARCHITECTURE

UNDERSTAND HOW YOUR SYSTEM WORKS IN PRODUCTION

1 ENTRY POINTS AND TRAFFIC FLOW

Where users come from (web, mobile, third-party integrations).

2 DNS

Resolves domain names to the correct endpoints.

3 CDN AND LOAD BALANCERS

CDN caches static content. Load balancers distribute traffic across instances.

4 APIS AND SERVICES

APIs route requests to microservices or backend services.

5 KUBERNETES OR COMPUTE LAYER

Runs containers or compute instances at scale.

6 DATABASES

Stores application persistent data.

7 CACHES

Speeds up responses and reduces load on databases.

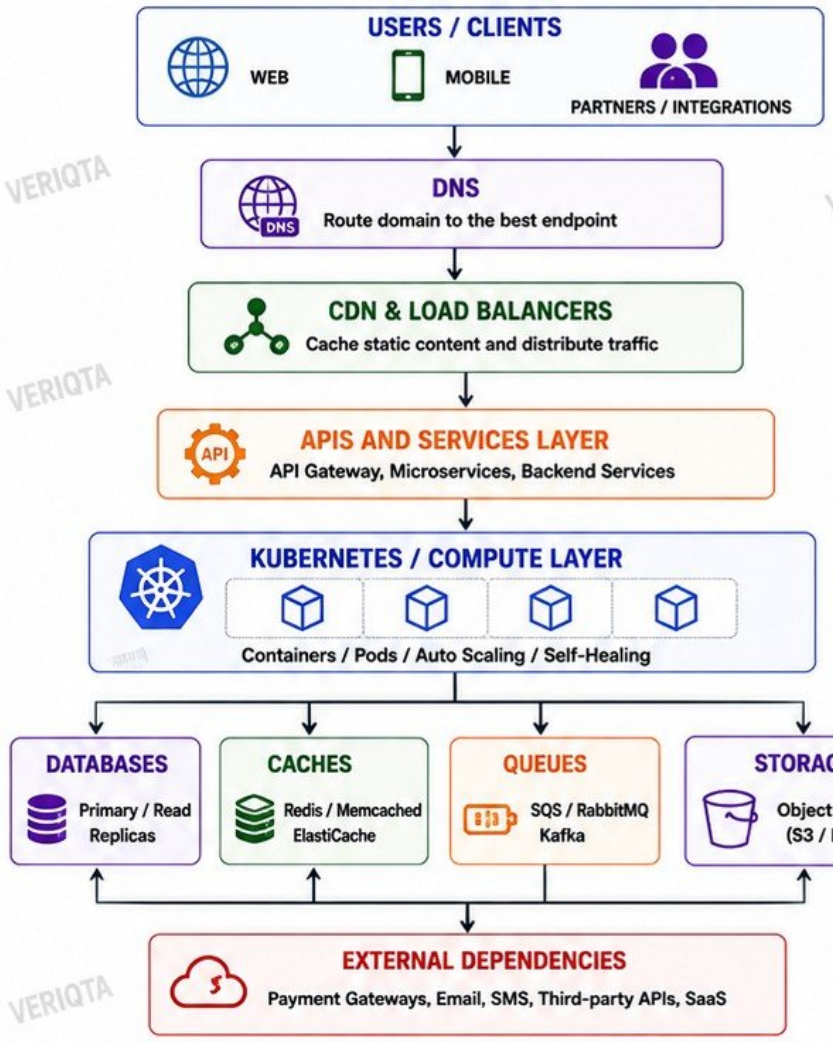
8 QUEUES

Decouple services and handle async processing.

9 EXTERNAL DEPENDENCIES

Third-party services your system relies on.

TYPICAL PRODUCTION ARCHITECTURE FLOW



10 BUILD YOUR OWN ARCHITECTURE MAP

- Draw the full request flow from user to data.
- Identify every component and its responsibility.
- Identify data stores, caches, queues, and dependencies.
- Understand the communication between services.
- Identify single points of failure.
- Ask: What happens if this component goes down?
- Identify scaling points.
- Identify security boundaries.
- Know who owns each component.
- Keep the map updated.
- Visualize before you optimize.
- Document everything clearly.

KEY PRINCIPLES

- Understand before you change.
- Observe real traffic and patterns.
- Security at every layer.
- Design for failure and scale.
- Collaborate with the right owners.

PRO TIP

A clear architecture map saves time, prevents incidents, and builds trust with your team.

YOUR GOAL

You should be able to explain the entire production flow on a whiteboard.

PAGE 4

UNDERSTAND THE CLOUD ENVIRONMENT

KNOW YOUR CLOUD BEFORE YOU BUILD OR CHANGE ANYTHING

1 AWS, AZURE, OR GCP ACCOUNT STRUCTURE

- Management account
- Workload accounts / Subscriptions / Projects
- Organizational units / Folders
- Control tower / Landing zone
- Understand the hierarchy and boundaries

2 REGIONS AND AVAILABILITY ZONES

- Regions = Physical locations
- AZs provide high availability
- Use multi-AZ for resilience
- Know where your data lives

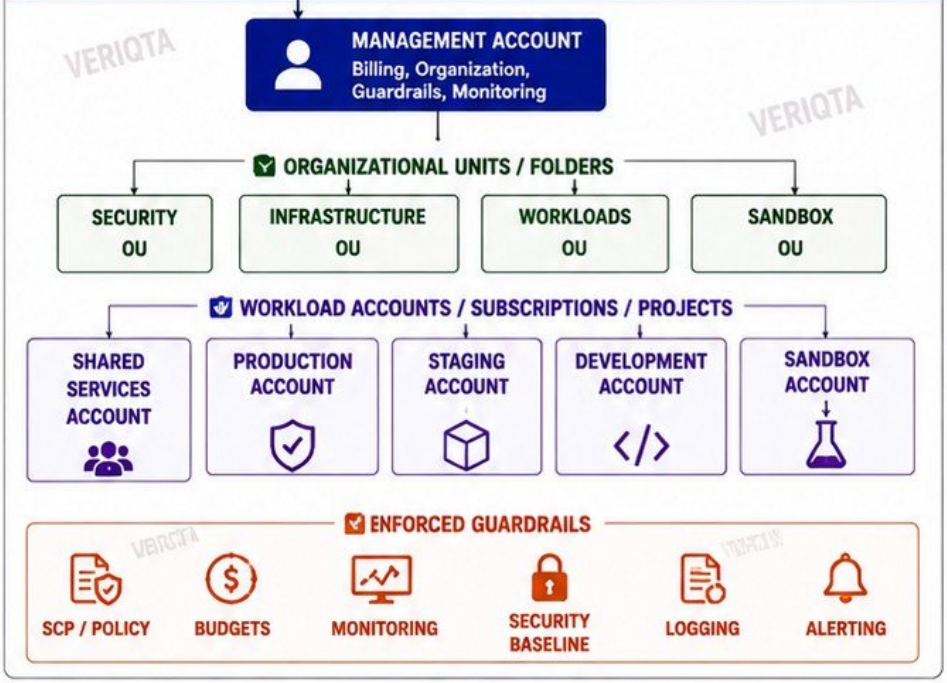
3 NETWORKING BOUNDARIES

- VPC / VNet / VPC
- Subnets (Public, Private, DB)
- Route tables and gateways
- Security Groups / NSGs
- NACLs and Firewall rules
- Peering, PrivateLink, VPN

4 SHARED SERVICES

- IAM / Identity management
- DNS management
- Monitoring & Logging
- Security services
- CI/CD, Artifact Repos
- Backup & DR services

TYPICAL CLOUD ACCOUNT STRUCTURE (EXAMPLE)



5 PRODUCTION VERSUS NON-PRODUCTION

PRODUCTION	NON-PRODUCTION
Handles real users	Dev, Test, QA, Staging
Highest availability	Lower cost optimized
Strict change control	More experimentation
Enhanced monitoring	Less strict controls
Backups & DR	Shorter data retention
Multi-AZ / Multi-Region	Used for validation

6 RESOURCE OWNERSHIP AND TAGGING

OWNERSHIP

- Every resource must have an owner
- Clear escalation
- Accountable teams
- Document contacts

TAGGING (EXAMPLES)

- Environment (Prod/Dev)
- Owner / Team
- Application
- Cost Center
- Criticality
- Data Classification

7 WHERE CRITICAL WORKLOADS ACTUALLY RUN

- Don't rely only on diagrams
- Verify in the cloud consoles
- Check regions and AZs
- Identify dependencies
- Know failover locations
- Understand data location

8 NEVER ASSUME THE ARCHITECTURE DIAGRAM IS CURRENT

- Systems change constantly
- Resources get added / removed
- Shadow IT happens
- Always validate before you act
- Trust, but verify

FIRST WEEK ACTION PLAN

- Get account access and explore
- Identify key teams and owners
- Understand regions and networks
- Review security controls and policies
- Find monitoring and logging setup
- Document what you learn

PAGE 5

LEARN THE INFRASTRUCTURE AS CODE

AUTOMATE. VERSION. REVIEW. REPEAT.

1. LOCATE IaC TOOLS

- TERRAFORM
- CLOUDFORMATION
- PULUMI
- OTHER IaC (ARM, CDK, BICEP, etc.)

FIND WHERE INFRASTRUCTURE IS DEFINED.
DO NOT ASSUME – CONFIRM.

2. REPOSITORY STRUCTURE (EXAMPLE)

```
iac-repo/
├── envs/
│   ├── prod/
│   ├── staging/
│   └── dev/                 → Environment configurations
├── modules/
│   ├── vpc/
│   ├── eks/
│   ├── database/
│   └── iam/                 → Reusable Infrastructure components
├── shared/
│   ├── providers.tf
│   └── backend.tf           → Shared configs and backends
├── README.md
└── Makefile                 → Documentation and automation
```

3. MODULES

- Break infrastructure into reusable modules
- Keep modules small and focused
- Use input variables and outputs
- Version modules
- Document module usage

4. STATE MANAGEMENT

- Store state remotely (S3, Azure Blob, GCS)
- Enable state locking (DynamoDB, Blob Lease, etc.)
- Encrypt state files
- Restrict access to state
- Never commit state to Git

5. ENVIRONMENT SEPARATION

PRODUCTION
- Highest reliability
- Strict change control
- Least privilege
- Separate state
- Separate accounts

STAGING
- Mirrors production
- Pre-production testing
- Separate state
- Isolated resources

DEVELOPMENT
- Fast iteration
- Lower cost
- Separate state
- Safe to break

6. VARIABLES AND SECRETS

- Use variables for all dynamic values
- Do not hardcode
- Store secrets in secure systems (AWS Secrets Manager, Vault, etc.)
- Use .tfvars files (not for secrets)
- Never expose secrets in code or logs

7. CI VALIDATION

- Format check (terraform fmt)
- Validate configuration (terraform validate)
- Lint code (tflint, checkov)
- Plan on every pull request
- Run policy as code (OPA, Sentinel)
- Fail fast before merge

8. CHANGE APPROVAL PROCESS

1. Create change (PR)
2. Automated checks pass
3. Peer review
4. Plan reviewed
5. Approval from code owners
6. Merge
7. Apply in controlled pipeline
8. Verify after apply

9. NEVER RUN CHANGES BLINDLY

- Always run plan first
- Review every change
- Understand the impact
- Apply only what you understand
- Small, safe changes
- Rollback plan ready

NEVER RUN INFRASTRUCTURE CHANGES BLINDLY

PLAN FIRST | REVIEW CAREFULLY | UNDERSTAND THE IMPACT | GET APPROVAL | APPLY SAFELY | VERIFY & DOCUMENT

FIRST WEEK ACTIONS

- Find the IaC repositories
- Understand repo structure
- Review modules and usage
- Confirm state management
- Understand variables and secrets
- Run validation locally
- Learn the approval process

PAGE 6

UNDERSTAND THE CI/CD PIPELINE

FROM CODE COMMIT TO PRODUCTION

1 CODE COMMIT
- Developer commits code
- Triggers pipeline
- Version control records change

2 PULL REQUEST CHECKS
- Code review
- Static code analysis
- Linting
- Policy checks
- PR approval required

GATE 1 PR APPROVAL

3 BUILD
- Compile code
- Resolve dependencies
- Run build scripts
- Fail fast on errors

4 UNIT & INTEGRATION TESTING
- Unit tests
- Integration tests
- Test reports
- Coverage reports

GATE 2 TESTS PASS

5 SECURITY SCANNING
- SAST / DAST
- Dependency scan
- Container scan
- Secrets scan
- Policy compliance

GATE 3 SECURITY PASS

6 ARTIFACT CREATION
- Package application
- Generate artifacts
- Version artifacts
- Store build metadata

7 CONTAINER REGISTRY
- Build container image
- Tag image
- Push to registry
- Immutable artifacts

GATE 4 IMAGE PUBLISHED

8 DEPLOYMENT
- Deploy to target environment
- IaC / GitOps / Helm
- Rolling / Blue-Green / Canary
- Update infrastructure

GATE 5 DEPLOY APPROVED

9 VERIFICATION
- Health checks
- Smoke tests
- Synthetic monitoring
- Business validation
- Metrics validation

GATE 6 VERIFICATION PASS

10 ROLLBACK
- Automatic rollback on failure
- Manual rollback option
- Previous version recovery
- Minimize impact

TYPICAL PIPELINE FLOW

CODE COMMIT → PULL REQUEST CHECKS → BUILD → TESTING → SECURITY SCAN → ARTIFACT → REGISTRY → DEPLOY → VERIFY → ROLLBACK

IDENTIFY EVERY PRODUCTION GATE

- **GATE 1: PR APPROVAL** — No unreviewed code enters the pipeline.
- **GATE 2: TESTS PASS** — All unit and integration tests must pass.
- **GATE 3: SECURITY PASS** — No high or critical vulnerabilities allowed.
- **GATE 4: IMAGE PUBLISHED** — Artifact is built, signed, and stored securely.
- **GATE 5: DEPLOY APPROVED** — Deployment requires manual or automated approval.
- **GATE 6: VERIFICATION PASS** — Application is healthy and functional.

BEST PRACTICES

- Make the pipeline fast, reliable, and repeatable.
- Fail early and provide clear feedback.
- Keep environments as similar as possible.
- Automate everything that can be automated.
- Use versioned, immutable artifacts.
- Monitor deployments like production changes.
- Always have a tested rollback plan.

A good pipeline builds confidence.

A bad pipeline creates risk.

FIRST WEEK ACTIONS

- Find the main CI/CD pipeline repository
- Understand who owns the pipeline at each stage
- List all pipeline stages and approvals
- Identify security scans and quality gates
- Understand artifact and deployment process
- Observe a real deployment from start to finish
- Understand the rollback process and triggers

PAGE 7

LEARN HOW PRODUCTION DEPLOYMENTS WORK

UNDERSTAND THE PROCESS. OBSERVE FIRST. OWN WITH CONFIDENCE.

1. DEPLOYMENT OWNERSHIP

- Someone owns every release
- Clear responsibility = successful deployments
- Know who approves, deploys, and validates
- Ownership prevents confusion and risk

2. RELEASE SCHEDULES

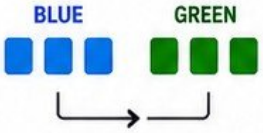
- Know the release calendar
- Understand freeze windows
- Align changes with business needs
- Avoid surprises and last-minute changes
- Plan, communicate, and stick to it

3. DEPLOYMENT STRATEGIES

ROLLING DEPLOYMENT

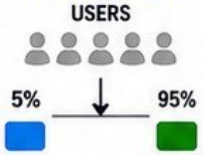
Update instances one at a time. Minimal downtime.

BLUE / GREEN



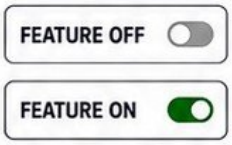
Switch traffic between two identical environments. Fast rollback.

CANARY DEPLOYMENT



Send a small % of traffic to new version. Validate before full rollout.

FEATURE FLAGS



Release code early. Enable features safely when ready.

4. MANUAL APPROVALS

- Human checkpoints add safety
- Understand what needs approval
- Never bypass required approvals

5. DEPLOYMENT VERIFICATION

- Verify after every deployment
- Check metrics, logs, and health checks
- Confirm functionality end-to-end

6. ROLLBACK PROCEDURES

- Know how to roll back and when
- Keep rollback simple and rehearsed
- Document and test your rollback plan

7. OBSERVE BEFORE OWNING ONE

- Watch the full process multiple times
- Ask questions
- Understand the "why" behind each step
- Confidence comes from observation

8. COMMUNICATION IS CRITICAL

- Communicate before, during, and after
- Share status and impact
- Document decisions
- Keep stakeholders informed

TYPICAL PRODUCTION DEPLOYMENT FLOW

KEY TAKEAWAY

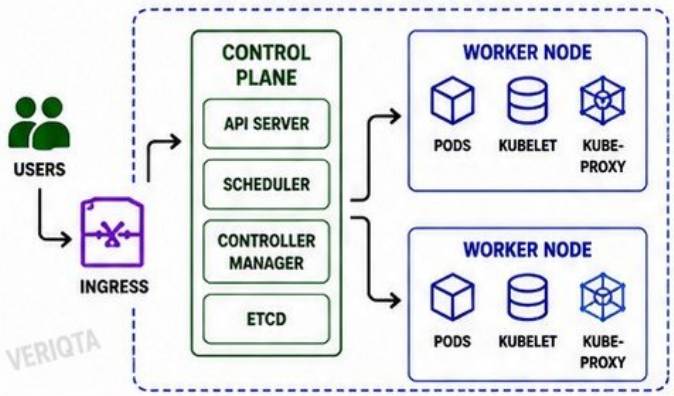
Production deployments are not just technical steps.
They are a process of planning, communicating, validating, and learning.
Observe first. Understand fully. Then own with confidence.

PAGE 8

UNDERSTAND KUBERNETES BEFORE TOUCHING IT

LEARN THE FOUNDATION. RESPECT THE CLUSTER.

1. CLUSTER ARCHITECTURE



THE CONTROL PLANE MAKES DECISIONS.
WORKER NODES RUN YOUR APPLICATIONS.

2. NAMESPACES

- Isolate environments and teams
- Manage resources and access
- Avoid putting everything in "default"

3. WORKLOADS

- Pods are the smallest deployable unit
- Deployments manage replica sets
- StatefulSets for stateful applications
- DaemonSets, Jobs, CronJobs, etc.

4. SERVICES

- Stable network identity for Pods
- ClusterIP, NodePort, LoadBalancer
- Enable service discovery and routing

5. INGRESS

- Manage external HTTP/HTTPS access
- Host and path based routing
- Use Ingress Controller (NGINX, ALB, etc.)

6. CONFIGMAPS & SECRETS

- ConfigMaps store non-sensitive configuration
- Secrets store sensitive data (passwords, keys, tokens)
- Never hardcode secrets
- Use environment variables or mounted files

7. RBAC

- Control who can do what
- Use Roles and RoleBindings within namespaces
- Use ClusterRoles and ClusterRoleBindings for cluster-wide access
- Principle of least privilege

8. RESOURCE REQUESTS AND LIMITS

- Requests = minimum resources needed
- Limits = maximum resources allowed
- Prevent noisy neighbor problems
- Enable proper scheduling and cost control

9. AUTOSCALING

- Horizontal Pod Autoscaler (HPA) scales Pods
- Cluster Autoscaler scales nodes
- Scale on CPU, memory, or custom metrics
- Test scaling before you need it

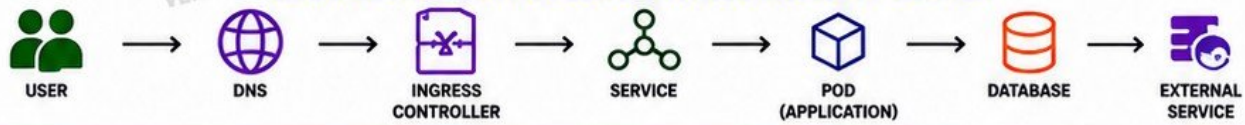
10. GITOPS OWNERSHIP

- Infrastructure and apps managed in Git
- Argo CD / Flux keep cluster in desired state
- Pull-based deployments
- Auditable, versioned, and repeatable

11. IDENTIFY CRITICAL WORKLOADS AND DEPENDENCIES

- Know what the business cannot afford to lose
- Map dependencies between services, databases, queues, and external systems
- Document owners and recovery strategies

EXAMPLE: HOW TRAFFIC FLOWS IN A KUBERNETES APPLICATION



KEY TAKEAWAY

- ✓ Understand before you change
- ✓ Observe the cluster in action
- ✓ Learn the core objects and how they work together
- ✓ Respect permissions and boundaries
- ✓ Map critical workloads and dependencies
- ✓ Ask questions, read docs, and practice safely

PAGE 9

LEARN THE NETWORKING PATH

UNDERSTAND HOW TRAFFIC MOVES FROM USER TO DATABASE.

1. DNS RESOLUTION

DNS

- Translates domain names to IP addresses
- Check TTL and caching
- Understand DNS provider and records (A, CNAME, MX)

2. VPC / VNET

- Isolated network boundary in the cloud
- CIDR blocks and address planning
- Understand peering and connections

3. SUBNETS

- Split network into smaller segments
- Public subnet: internet facing resources
- Private subnet: backend resources

4. ROUTING

- Route tables control where traffic goes
- Understand local, static, and propagated routes
- Always check the effective route for a resource

5. NAT

NAT

- Allows private resources to access the internet
- NAT Gateway for high availability
- Understand inbound vs. outbound traffic

6. FIREWALLS & SECURITY GROUPS

- Control inbound and outbound traffic
- Use least privilege
- Stateless (SG) vs. stateful (Firewall) rules

7. LOAD BALANCERS

- Distribute traffic across multiple targets
- Types: ALB, NLB, GLB
- Health checks ensure only healthy targets receive traffic

8. KUBERNETES NETWORKING

- CNI plugin provides Pod networking
- Services provide stable endpoints
- Ingress manages external access
- Network Policies control traffic between Pods

9. SERVICE-TO-SERVICE COMMUNICATION

- Internal traffic stays within the cluster
- Use Services, not Pod IPs
- mTLS and Network Policies for security
- Observe latency and retries

10. TRACE ONE REQUEST FROM USER TO DATABASE

1. **USER** — User enters domain in browser
2. **DNS** — DNS resolves domain to public IP
3. **INTERNET** — Request travels over the internet
4. **LOAD BALANCER** — Traffic hits Load Balancer (ALB / NLB)
5. **INGRESS CONTROLLER** — Ingress routes traffic to the correct service
6. **SERVICE** — Kubernetes Service forwards to Pods
7. **APPLICATION POD** — Pod processes the request (app logic)
8. **DATABASE** — App connects to Database (private subnet)

PUBLIC NETWORK → KUBERNETES CLUSTER → PRIVATE NETWORK

CHECKLIST: THINGS TO VERIFY

- [x] DNS records are correct and not expired
- [x] Route tables send traffic to the right destinations
- [x] Security groups/NACLs allow required traffic
- [x] Load balancer health checks are passing
- [x] Ingress rules are correct
- [x] Pods are healthy and ready
- [x] Database is reachable from application pods
- [x] Logs and metrics show smooth request flow

COMMON TOOLS TO USE

- nslookup / dig
- traceroute / mtr
- curl / wget
- VPC Flow Logs
- kubectl get all
- kubectl describe svc / pod / ingress
- CloudWatch / Prometheus / Grafana

KEY TAKEAWAY

Know the path.
Know the controls.
Know where it can fail.

TRACE IT. UNDERSTAND IT. FIX IT FASTER.

PAGE 10

LEARN
IAM, SECRETS, AND PRODUCTION ACCESS

SECURE ACCESS. PROTECT SECRETS. EARN TRUST.

1. IAM USERS AND ROLES

- Users are for people
- Roles are for permissions
- Avoid long-term access keys
- Use Federated access (SSO)
- Rotate credentials regularly

2. SERVICE IDENTITIES

- Use roles for services and workloads
- EC2, ECS, EKS, Lambda, etc.
- No hardcoded credentials
- Use OIDC / IRSA for Kubernetes

3. LEAST PRIVILEGE

- Grant minimum permissions needed
- Start small, add only what is needed
- Review and remove unused permissions

4. PRODUCTION ACCESS

- Access production only when needed
- Use SSO with MFA
- Separate accounts for dev, staging, and prod
- No direct access to resources

5. PRIVILEGED ESCALATION

- Use short-lived elevated access
- Approvals and justification required
- All elevation should be logged
- No standing admin access

6. SECRETS MANAGEMENT

- Store secrets in a secrets manager
- Never in code, configs, or env files
- Rotate secrets automatically
- Use dynamic credentials where possible

7. KMS AND ENCRYPTION

- Encrypt data at rest and in transit
- Use KMS for key management
- Use customer managed keys for sensitive data
- Control key access tightly

8. AUDIT TRAILS

- Enable CloudTrail / Activity Logs
- Monitor all API calls
- Send logs to central account
- Review logs regularly
- Detect unusual activity early

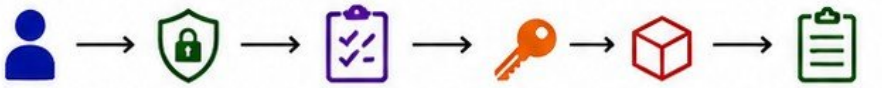
9. BREAK-GLASS ACCESS

- Emergency access for critical situations only
- Store credentials securely
- Access is audited and reviewed
- Test the break-glass process

10. NEVER USE PRODUCTION ACCESS CASUALLY

- Treat production like it is live, because it is
- Is think before you run any command
- Small mistakes can cause big outages
- Protect the business, protect your access

PRODUCTION ACCESS MODEL (EXAMPLE)



USER	SSO + MFA	ROLE ASSUMPTION	TEMPORARY CREDENTIALS	PRODUCTION RESOURCES	AUDIT LOGS
Engineer	Authenticate	Assume least privilege role	Short-lived credentials	Access what you need	All actions are recorded

ACCESS CHECKLIST

- ☑ Do I really need access?
- ☑ Am I using MFA / SSO?
- ☑ Is this the least privileged role?
- ☑ Is access time-limited?
- ☑ Are my actions logged?
- ☑ Did I verify before I changed anything?
- ☑ Will someone review if something goes wrong?

- ✓ Use Roles, not long-term access keys
- ✓ Follow least privilege at all times
- ✓ Store secrets in a secrets manager
- ✓ Enable MFA for all human access
- ✓ Monitor and review access regularly
- ✓ Rotate credentials and secrets
- ✓ Encrypt sensitive data always
- ✓ Use separate accounts for environments
- ✓ Document access and ownership
- ✓ Review and remove unused access

COMMON TOOLS

- AWS IAM
- AWS SSO / IAM Identity Center
- AWS Secrets Manager
- AWS Systems Manager Parameter Store
- AWS KMS
- AWS CloudTrail
- AWS Config

REMEMBER

ACCESS IS A PRIVILEGE. NOT A RIGHT.

YOU ARE TRUSTED WITH PRODUCTION FOR A REASON. PROTECT THAT TRUST.

PAGE 11

UNDERSTAND MONITORING AND OBSERVABILITY

YOU CAN'T IMPROVE WHAT YOU DON'T OBSERVE.

1. METRICS

- Numerical data over time
- System and application health
- Trends and anomalies
- Use RED / USE method

2. LOGS

- Detailed events with context
- Essential for troubleshooting
- Search, filter, and correlate
- Centralize and retain properly

3. TRACES

- Track requests end-to-end
- See latency across services
- Identify slow dependencies
- Correlate across microservices

4. DASHBOARDS

- Visualize what matters
- Real-time awareness
- Overview and deep-dive views
- Keep it simple and useful

5. ALERTS

- Notify on the right conditions
- Actionable and owned
- Avoid alert fatigue
- Route to the right people

6. SLI, SLO, AND SLA

SLI
Service Level Indicator
What you measure.
Example: API latency, error rate, uptime.

SLO
Service Level Objective
The target you aim for.
Example: 99.9% uptime, p95 < 300ms.

SLA
Service Level Agreement
The promise you make.
Example: 99.9% uptime in contract.

7. APPLICATION VS. INFRASTRUCTURE MONITORING

VS.

Application	Infrastructure
• User experience	• CPU, memory, disk
• Transactions	• Network, IO
• Business metrics	• Containers, nodes
• Errors and latency	• Availability

8. FIND THE DASHBOARDS ENGINEERS ACTUALLY USE

- Ask the team
- Don't assume
- Learn the go-to views
- Understand key panels
- Save and organize what matters

9. IDENTIFY BLIND SPOTS BEFORE ADDING NEW ALERTS

- What isn't monitored?
- What can fail silently?
- Where are the gaps?
- Validate with real incidents
- Close gaps before adding alerts

10. GOLDEN RULE

Monitor for understanding.
Alert for action.
Observe continuously.
Improve always.

THE OBSERVABILITY STACK

INSTRUMENT → Add telemetry (metrics, logs, traces)

COLLECT → Agents, exporters, and collectors

STORE → Time-series DB, log store

VISUALIZE → Dashboards and queries

ALERT → Notify the right people

LEARN & IMPROVE → Reduce MTTR, improve reliability

KEY TAKEAWAY

GOOD OBSERVABILITY IS PROACTIVE, NOT REACTIVE.
UNDERSTAND THE SYSTEM. KNOW THE NORMAL. FIND THE UNKNOWN.

PAGE 12

LEARN HOW THE TEAM HANDLES INCIDENTS

BE PREPARED. RESPOND CALMLY. SOLVE THE RIGHT PROBLEM.

1. INCIDENT SEVERITY LEVELS

SEV 1 CRITICAL
- Major outage
- Severe impact
- Business down

SEV 2 HIGH
- Significant impact
- Degraded service
- Workarounds exist

SEV 3 MEDIUM
- Minor impact
- Limited users
- Non-urgent

SEV 4 LOW
- Minimal impact
- Cosmetic or minor
- Informational

2. ON-CALL PROCESS

You are alerted
↓
Acknowledge the alert
↓
Assess and engage team
↓
Resolve or escalate
↓
Document and close the incident

3. ALERT ROUTING

MONITORING SYSTEM → ALERT ROUTER → ON-CALL SCHEDULE → ENGINEER NOTIFIED

- Alerts should reach the right person, always
- Follow the routing, don't bypass it
- Escalate if no acknowledgment
- Keep alert noise low and meaningful

4. INCIDENT COMMANDER

- One person leads the incident
- Coordinates the response
- Assigns tasks
- Communicates updates
- Makes escalation decisions
- Keeps focus on resolution

5. COMMUNICATION CHANNELS

SLACK / TEAMS
Real-time updates

VOICE CALLS
For critical coordination

EMAIL
For status and follow-ups

INCIDENT CHANNEL
Single source of truth

6. ESCALATION

LEVEL 1 ON-CALL ENGINEER
LEVEL 2 SENIOR ENGINEER
LEVEL 3 TEAM LEAD / MANAGER
LEVEL 4 DIRECTOR / EXECUTIVE

- Escalate early, not late
- Escalate based on impact, not ego
- Clear escalation paths save time
- Know who to contact and when

7. MITIGATION VS. ROOT-CAUSE RESOLUTION

MITIGATION (SHORT TERM)
- Restore service
- Reduce impact
- Get users unblocked
- Example: Restart service, scale up, rollback

ROOT-CAUSE (LONG TERM)
- Find the real cause
- Fix the underlying issue
- Prevent recurrence
- Example: Code fix, config change, test

Solve the symptom to stop the pain.
Solve the cause to prevent it from coming back.

8. POSTMORTEMS

- Blameless culture
- What happened?
- Impact
- Root cause
- What went well?
- What didn't?
- Action items
- Owners & due dates
- Share learnings
- Track to completion
- Improve the system, not just the fix

9. READ PREVIOUS INCIDENTS BEFORE JOINING ON-CALL

- Read recent postmortems
- Understand common failure patterns
- Learn the system's weak points
- Know what has already been tried
- Avoid repeating mistakes
- Be prepared before the alert comes

KEY TAKEAWAY

INCIDENTS WILL HAPPEN.
HOW THE TEAM RESPONDS IS WHAT PROTECTS USERS AND BUILDS TRUST.

PAGE 13

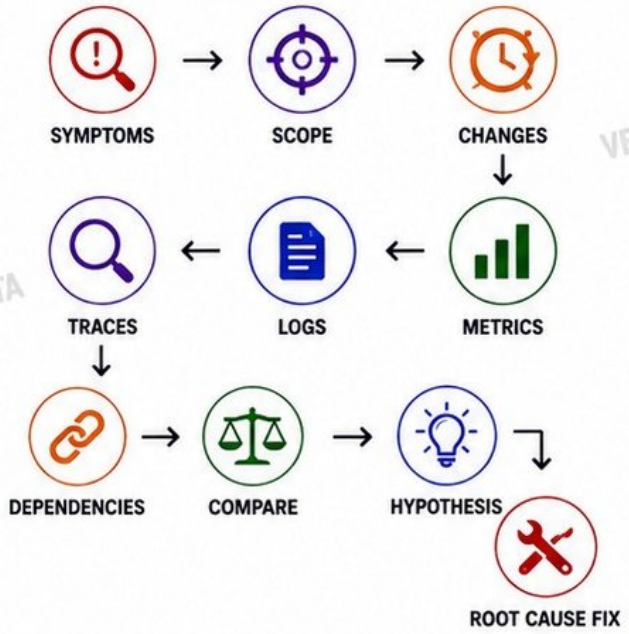
MASTER PRODUCTION TROUBLESHOOTING

A SYSTEMATIC APPROACH. NO GUESSWORK.

10-STEP TROUBLESHOOTING FRAMEWORK

1. **START WITH SYMPTOMS**
   What is the issue? What are users experiencing? Gather facts, not opinions.
2. **ESTABLISH SCOPE AND BLAST RADIUS**
   How many users are affected? Which regions, services, or components are impacted?
3. **CHECK RECENT CHANGES**
   Deployments, configs, infrastructure, tickets, database changes. Correlate time.
4. **INSPECT METRICS**
   Check dashboards and key metrics. Look for anomalies and trends.
5. **INSPECT LOGS**
   Search logs with the right time range. Look for errors, warnings, and patterns.
6. **FOLLOW TRACES**
   Trace requests end-to-end. Identify latency and failure points.
7. **VALIDATE DEPENDENCIES**
   Check downstream services, databases, queues, third-party APIs, and networks.
8. **COMPARE HEALTHY AND UNHEALTHY SYSTEMS**
   Find the differences. Look at configs, versions, resource usage, and behavior.
9. **TEST HYPOTHESES**
   Form a theory. Test it safely. Prove it right or wrong.
10. **FIX THE ROOT CAUSE, NOT THE SYMPTOM**
    Implement a proper fix. Validate. Prevent recurrence. Document what you learned.

TROUBLESHOOTING FLOW



GOOD TROUBLESHOOTING HABITS

- Stay calm. Gather facts.
- Don't jump to conclusions.
- Change one thing at a time.
- Document everything.
- Communicate clearly.
- Think systemically, not in silos.

THINGS TO AVOID

- Rebooting everything without understanding
- Changing things blindly
- Ignoring the basics
- Tunnel vision
- Fixing the symptom
- Ignoring user impact

USE THE RIGHT TOOLS

METRICS	LOGS	TRACES	ALERTS	INFRASTRUCTURE	NETWORK
CloudWatch	CloudWatch Logs	AWS X-Ray	PagerDuty	AWS Console	Ping, Curl
Prometheus	ELK / OpenSearch	Jaeger	Opsgenie	Kubectl	Traceroute
Grafana	Loki	Zipkin	Slack	Terraform	DNS Lookup

KEY TAKEAWAY

THE BEST ENGINEERS AREN'T THE ONES WHO KNOW EVERY TOOL.
THEY ARE THE ONES WHO ASK THE RIGHT QUESTIONS IN THE RIGHT ORDER.

PAGE 14

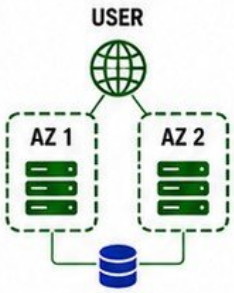
UNDERSTAND RELIABILITY AND FAILURE MODES

DESIGN FOR FAILURE. DELIVER RELIABILITY.

1 SINGLE POINTS OF FAILURE

- A single component that, if it fails, stops the entire system.
- Identify and eliminate them.

2 MULTI-AZ ARCHITECTURE



- Distribute across Availability Zones.
- Survive datacenter failures.

3 REDUNDANCY

- Duplicate critical components.
- No single component should be critical.

4 HEALTH CHECKS

- Continuously verify components are healthy.
- Remove unhealthy instances automatically.

5 AUTO SCALING

- Scale out with demand.
- Scale in when demand decreases.
- Right size for reliability and cost.

6 TIMEOUTS

- Never wait forever.
- Set reasonable timeouts for every operation.
- Fail fast and move on.

7 RETRIES

- Retry transient failures with backoff.
- Avoid retry storms.
- Use exponential backoff with jitter.

8 CIRCUIT BREAKERS

- Stop sending requests when a service is failing.
- Allow time to recover.
- Prevent cascading failures.

9 GRACEFUL DEGRADATION

- Reduce functionality, not availability.
- Prioritize critical features.
- Keep the system useful under stress.

10 DISASTER RECOVERY

- Have a plan for worst-case scenarios.
- Define RTO and RPO.
- Test recovery regularly.

THINK ABOUT FAILURE

ASK THIS BEFORE EVERY DESIGN AND CHANGE:
"WHAT HAPPENS WHEN THIS COMPONENT FAILS?"

WHAT CAN FAIL?

WHO IS AFFECTED?

HOW WILL WE DETECT IT?

HOW WILL WE RECOVER?

HOW WILL WE PREVENT IT NEXT TIME?

KEY TAKEAWAY

RELIABILITY IS NOT AN ACCIDENT.
IT IS DESIGNED, BUILT, TESTED, AND CONTINUOUSLY IMPROVED.

PAGE 15

UNDERSTAND BACKUPS AND DISASTER RECOVERY

PREPARE TODAY. RECOVER TOMORROW.

1 WHAT IS BACKED UP

- Databases
- Application data
- File storage
- Configurations
- Infrastructure as code
- Secrets and keys

2 BACKUP FREQUENCY

- Real-time
- Every 15 minutes
- Hourly
- Daily
- Weekly
- Define based on criticality

3 RETENTION

- How long backups are kept
- 7 days / 30 days / 90 days / 1 year+
- Balance compliance, cost, and needs

4 RTO (RECOVERY TIME OBJECTIVE)

- Maximum acceptable downtime
- Example: 1 hour
- Drives architecture, automation, and testing

5 RPO (RECOVERY POINT OBJECTIVE)

- Maximum acceptable data loss
- Example: 15 minutes
- Determines backup frequency and replication

6 RESTORE PROCEDURES

- Documented step-by-step restore runbooks
- Easy to follow under pressure
- Keep runbooks updated and tested

7 CROSS-REGION RECOVERY

- Replicate to another Region
- Protects against regional disasters
- Verify replication and consistency

8 DATABASE RECOVERY

- Point-in-time restore
- Transaction logs
- Consistency checks
- Test restores on a regular basis

9 RECOVERY OWNERSHIP

- Who owns backups and restores
- Who approves recovery
- Who communicates during a disaster
- Clear roles and contacts

10 A BACKUP IS NOT PROVEN UNTIL RESTORATION IS TESTED

- Test restore regularly
- Validate data integrity
- Run disaster recovery drills
- Improve continuously

BACKUP AND DR RECOVERY OVERVIEW

KEY TAKEAWAY

BACKUPS GIVE YOU OPTIONS.
TESTED RESTORES GIVE YOU CONFIDENCE.

PAGE 16

FIND TECHNICAL DEBT WITHOUT BECOMING THE NEW HIRE WHO CHANGES EVERYTHING

OBSERVE. UNDERSTAND. PRIORITIZE. IMPROVE.

1 OUTDATED DEPENDENCIES

- Old libraries and frameworks
- Security vulnerabilities
- No longer maintained
- Harder to integrate

2 MANUAL PROCESSES

- Repetitive tasks done by people
- Prone to human error
- Slow and inconsistent
- Candidates for automation

3 FRAGILE PIPELINES

- Frequent failures
- Long and complex pipelines
- Poor feedback loops
- Hard to trust deployments

4 CONFIGURATION DRIFT

- Environments are inconsistent
- Changes not version controlled
- Manual changes in production
- Hard to reproduce issues

5 MISSING MONITORING

- No visibility into key metrics
- Blind spots in systems
- Issues only found by users
- Can't prove reliability

6 OVERPRIVILEGED ACCESS

- More access than needed
- High security risk
- Hard to audit
- Violates least privilege

7 UNUSED INFRASTRUCTURE

- Idle resources wasting money
- Old environments not decommissioned
- IPs, disks, snapshots, and more
- Clean up regularly

8 POOR DOCUMENTATION

- Missing or outdated documentation
- Hard for new hires to onboard
- Knowledge silos
- Tribal knowledge risk

9 REPEATED INCIDENTS

- Same issues happening again
- Root causes not fixed
- Workarounds instead of solutions
- Erodes reliability and trust

10 RANK BY RISK & BUSINESS IMPACT

- High risk + high impact = fix first
- Low effort + high impact = quick wins
- Align with business priorities
- Communicate value

HOW TO APPROACH TECHNICAL DEBT THE RIGHT WAY

OBSERVE
Listen, learn, and understand the current state.

ASK WHY
Understand the history and context.

IDENTIFY
Spot real problems, not just things you don't like.

EVALUATE
Assess risk, impact, effort, and urgency.

PROPOSE
Suggest small, safe, incremental improvements.

DELIVER VALUE
Improve iteratively and build trust over time.

GOLDEN RULE

UNDERSTAND WHY SOMETHING EXISTS BEFORE YOU TRY TO CHANGE IT.
RESPECT THE SYSTEM. THEN MAKE IT BETTER.

VERIQTA

/VERIQTA /VERIQTA /VERIQTA /VERIQTA /VERIQTA /VERIQTA

PAGE 17

DELIVER YOUR FIRST SAFE IMPROVEMENT

SMALL WINS BUILD TRUST. SAFE CHANGES BUILD IMPACT.

1 CHOOSE A LOW-RISK, HIGH-VALUE PROBLEM

- Solve a real problem for users or the team
- Low blast radius
- High impact or high visibility
- Quick to validate

2 ESTABLISH THE CURRENT STATE

- Gather baseline metrics and data
- Understand the existing workflow
- Identify pain points and constraints

3 DEFINE EXPECTED IMPACT

- What problem will this solve?
- How will we measure success?
- Define clear and realistic goals

4 TEST OUTSIDE PRODUCTION

- Test in dev/staging environment
- Validate functionality
- Run performance and integration tests
- Break it. Fix it.

5 PEER REVIEW

- Share your plan and solution
- Get feedback early
- Improve before requesting change
- Learn from others

6 CHANGE APPROVAL

- Follow the change management process
- Provide all necessary details
- Get approval from the right people

7 DEPLOYMENT PLAN

- Define step-by-step deployment steps
- Include prerequisites and dependencies
- Decide deployment strategy (e.g., rolling)
- Communicate the plan

8 ROLLBACK PLAN

- Define rollback steps
- Ensure you can revert quickly and safely
- Test the rollback process
- Know your exit criteria

9 PRODUCTION VERIFICATION

- Monitor metrics closely
- Validate functionality in production
- Confirm no negative impact
- Stay observant

10 DOCUMENT THE RESULT

- Document what was done and why
- Capture results and metrics
- Share learnings with the team
- Update runbooks if needed

PRINCIPLES OF A SAFE IMPROVEMENT

BE SAFE
Protect stability and users.

BE COLLABORATIVE
Work with the team, not around them.

BE TRANSPARENT
Communicate early and clearly.

BE MEASURABLE
Measure impact, not opinions.

BE ITERATIVE
Improve continuously in small steps.

KEY TAKEAWAY

YOUR FIRST IMPROVEMENT IS NOT ABOUT BEING FAST.
IT IS ABOUT BEING SAFE, THOUGHTFUL, AND REPEATABLE.

AUTOMATE ONE PAINFUL MANUAL PROCESS

BUILD RELIABLE AUTOMATION. ELIMINATE MANUAL RISK.

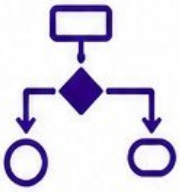
1 FIND REPETITIVE OPERATIONAL WORK

- Look for tasks done repeatedly
- Check runbooks, tickets, and alerts
- Talk to the team doing the work
- Choose something painful, not perfect

2 MEASURE FREQUENCY AND FAILURE RISK

- How often is it done?
- How long does it take?
- How often does it fail?
- What is the impact of failure?
- Prioritize by risk × frequency

3 DESIGN SAFE AUTOMATION



- Map the current process
- Define inputs, outputs and steps
- Keep it simple and clear
- Plan for safety at every step

4 MAKE OPERATIONS IDEMPOTENT

- Safe to run multiple times
- Prevent duplicate changes
- Use unique keys, checks, and states
- Idempotency reduces risk

5 ADD VALIDATION

- Validate inputs before execution
- Verify expected changes
- Fail fast on bad input
- Confirm success at the end

6 HANDLE FAILURES

- Expect failures
- Handle errors gracefully
- Add retries with backoff when safe
- Send alerts on failure
- Do not ignore errors

7 ADD LOGGING

- Log what happened
- Log inputs, outputs and important decisions
- Use structured logging
- Logs are for humans and automation

8 ADD ROLLBACK WHERE NECESSARY

- Plan how to undo changes
- Store previous state when needed
- Test rollback procedures
- Automate rollback when possible

9 DOCUMENT OWNERSHIP

- Document purpose, steps and usage
- Define owner and reviewers
- Keep documentation up to date
- Share with the team

10 AUTOMATION SHOULD REDUCE RISK, NOT HIDE COMPLEXITY

- Keep it transparent
- Keep it observable
- Keep it simple
- True automation makes things safer and easier

AUTOMATION SUCCESS PRINCIPLES

SAFETY FIRST
Never automate chaos.

VISIBILITY ALWAYS
If you can't see it, you can't trust it.

SIMPLE IS STRONG
Simple automation is reliable automation.

TEAM ownership
Automation that the team understands will last.

MEASURE IMPACT
Track time saved, errors reduced, risk reduced.

AUTOMATION SHOULD GIVE YOU TIME BACK AND REDUCE RISK.
IT SHOULD MAKE YOUR SYSTEM STRONGER, NOT HARDER TO UNDERSTAND.

DAYS 60 TO 90
START OWNING PRODUCTION

STEP UP. TAKE OWNERSHIP. DELIVER RELIABILITY.

1 HANDLE ROUTINE DEPLOYMENTS

- Deploy with confidence
- Follow the process
- Verify success
- Rollback if needed
- Communicate clearly

2 TROUBLESHOOT INDEPENDENTLY

- Diagnose issues on your own
- Use metrics, logs, and traces
- Find the root cause
- Fix problems end-to-end

3 PARTICIPATE IN INCIDENTS

- Join on-call rotations
- Respond quickly
- Communicate effectively
- Learn from every incident
- Help drive resolution

4 REVIEW INFRASTRUCTURE CHANGES

- Understand the proposed change
- Identify risks and impacts
- Question assumptions
- Ensure safe implementation

5 IMPROVE MONITORING

- Refine dashboards
- Improve alert quality
- Add useful metrics
- Reduce noise
- Increase visibility

6 CONTRIBUTE TO AUTOMATION

- Automate repetitive tasks
- Build scripts and tools
- Improve pipelines
- Reduce manual work
- Increase consistency

7 UPDATE RUNBOOKS

- Keep runbooks accurate
- Add what you learn
- Simplify steps
- Include examples
- Make them easy to follow

8 UNDERSTAND BUSINESS-CRITICAL SERVICES

- Know what matters most
- Map dependencies
- Understand SLIs and SLOs
- Protect customer impact

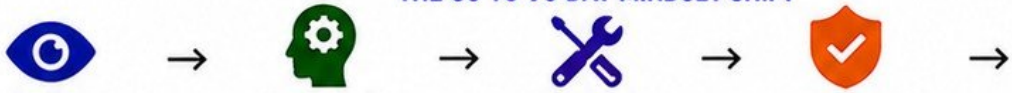
9 KNOW WHEN TO ESCALATE

- Escalate early
- Provide clear context
- Involve the right people
- Don't wait too long
- Protect the business

10 MOVE FROM OBSERVER TO RELIABLE CONTRIBUTOR

- Take ownership
- Deliver results
- Build trust
- Keep learning
- Make a positive impact every day

THE 60 TO 90 DAY MINDSET SHIFT



SUCCESS METRICS

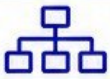
- ✓ DEPLOYMENTS: SUCCESSFUL AND SAFE
- ✓ INCIDENTS: RESPOND AND RESOLVE EFFECTIVELY
- ✓ IMPROVEMENTS: SMALL WINS, BIG IMPACT
- ✓ TRUST: YOU ARE RELIED ON

PAGE 20

DAY 90

PRODUCTION ENGINEER READINESS CHECK

CAN YOU OWN PRODUCTION WITH CONFIDENCE?

1		CAN YOU EXPLAIN THE ARCHITECTURE? ☐
2		CAN YOU TRACE A REQUEST END TO END? ☐
3		CAN YOU EXPLAIN HOW CODE REACHES PRODUCTION? ☐
4		CAN YOU SAFELY DEPLOY AND ROLL BACK? ☐
5		CAN YOU INVESTIGATE A PRODUCTION INCIDENT? ☐
6		CAN YOU FIND THE CORRECT LOGS AND METRICS? ☐
7		CAN YOU IDENTIFY CRITICAL DEPENDENCIES? ☐
8		CAN YOU EXPLAIN THE DISASTER RECOVERY STRATEGY? ☐
9		CAN YOU MAKE INFRASTRUCTURE CHANGES SAFELY? ☐
10		DO YOU KNOW WHAT YOU SHOULD NOT CHANGE? ☐
11		DO TEAMMATES TRUST YOU WITH PRODUCTION? ☐
12		ARE YOU READY TO OWN PRODUCTION? ☐

90-DAY PROGRESSION

DAYS 1 TO 30
UNDERSTAND

- ARCHITECTURE
- PEOPLE
- INFRASTRUCTURE
- PIPELINES
- SECURITY
- OPERATIONS

→

DAYS 31 TO 60
CONTRIBUTE

- TROUBLESHOOTING
- DEPLOYMENTS
- DOCUMENTATION
- SMALL FIXES
- MONITORING
- AUTOMATION

→

DAYS 61 TO 90
OWN

- PRODUCTION CHANGES
- INCIDENT PARTICIPATION
- INFRASTRUCTURE IMPROVEMENTS
- RELIABILITY WORK
- OPERATIONAL AUTOMATION
- INDEPENDENT TROUBLESHOOTING

SUCCESS IS NOT A DESTINATION. IT IS A PRACTICE.
KEEP LEARNING. KEEP IMPROVING. KEEP DELIVERING VALUE.